

Relatório técnico da entrega

Assistente médico acadêmico com LLM local fine-tuned via MLX, orquestração LangGraph, API Random Forest para risco em contexto de mamografia e interface Flutter Web.

AUTOR	Igor Hadson Leal Carletti
RM	371634
AUTOR	Paulo Eduardo Oliveira Moreira
RM	373862

REPOSITÓRIO
https://github.com/igorlealc/ia_tech_challenge_03
VÍDEO DA APRESENTAÇÃO
https://www.youtube.com/watch?v=zKze2eq5-9Q

Há, no repositório, uma versão deste arquivo em HTML, disponível na pasta raiz com o nome relatorio_entrega_fase3.html. Recomenda-se, preferencialmente, a consulta à versão em HTML, pois ela permite uma interação mais rica, além de melhor experiência de leitura, navegação e visualização das tabelas.

REPOSITÓRIO	BACKEND IA	MODELO PREDITIVO	FRONTEND
trabalho3/	FastAPI + LangGraph + MLX	RandomForestClass ifier	Flutter Web + Nginx

Resumo Executivo

A entrega da Fase 3 implementa uma aplicação local integrada para apoiar, em caráter acadêmico, a interpretação de casos clínicos relacionados a rastreamento ou investigação mamária. O usuário interage em linguagem natural por um chat Flutter Web; o backend organiza o fluxo com LangGraph, consulta uma LLM local com adapter LoRA e aciona uma API de inferência quando há dados suficientes para estimar a classe positiva do alvo `TARGET_CANCER_MAMA_PROVAVEL`.

O sistema não realiza diagnóstico, não prescreve condutas e não substitui avaliação médica. Todas as respostas incluem aviso de uso acadêmico e o fluxo registra auditoria local para rastreabilidade.

Python

FastAPI

LangGraph

LangChain

MLX

LoRA

Flutter Web

Docker Compose

Aderência aos Requisitos do Enunciado

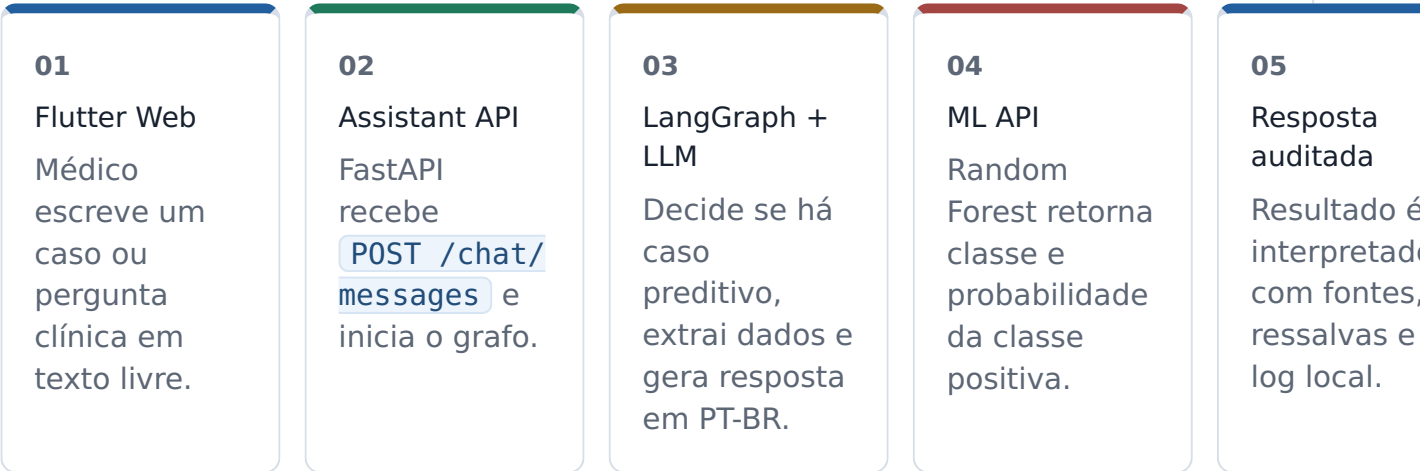
O arquivo [8IADT - Fase 3 - Tech challenge.pdf](#) define como requisitos: fine-tuning de LLM com dados médicos, criação de assistente com LangChain, segurança e validação, logging, explainability, organização modular em Python e README completo.

Requisito obrigatório	Implementação na entrega	Arquivos principais
Fine-tuning de LLM com dados médicos	Preparação de dataset PubMedQA em formato JSONL e treinamento de adapter LoRA com <code>mlx_lm.lora</code> .	<code>fine-tuning/1_preparar_dataset.py</code> , <code>fine-tuning/2_treinar_modelo.py</code> , <code>fine_tuning.sh</code>
Assistente médico com LangChain	API FastAPI com fluxo LangGraph, prompts clínico-acadêmicos, chamada à LLM local e ferramenta para consultar modelo preditivo.	<code>assistant/app/graph.py</code> , <code>assistant/app/tools.py</code> , <code>assistant/app/main.py</code>
Base estruturada e contexto atualizado	Integração com a API de ML que serve o modelo Random Forest treinado no trabalho anterior, usando 24 features codificadas.	<code>ml/app/model_service.py</code> , <code>ml/model/random_forest.joblib</code>
Segurança, validação e limites de atuação	Prompt de sistema restringe diagnóstico e prescrição; respostas recebem aviso acadêmico obrigatório; schemas Pydantic validam a entrada do chat.	<code>assistant/app/safety.py</code> , <code>assistant/app/schemas.py</code>
Logging e auditoria	Registro em JSONL com mensagem, decisão de uso do modelo, payload clínico, features, resultado da ferramenta, fontes e erros.	<code>assistant/app/audit.py</code> , <code>assistant/logs/audit.jsonl</code>
	README principal descreve execução, arquitetura,	

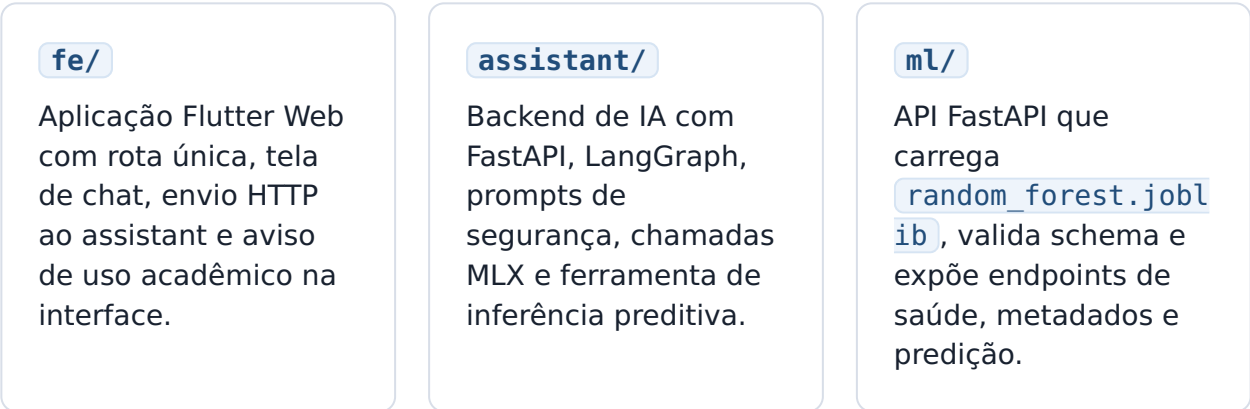
Requisito obrigatório	Implementação na entrega	Arquivos principais
README e código modularizado	endpoints, fine-tuning, Docker, artefatos e observações de segurança.	<code>README.md</code> , <code>ml/README.md</code> , <code>fe/README.md</code>

Arquitetura da Solução

A arquitetura separa a experiência de chat, a orquestração da IA e a inferência supervisionada. Essa divisão facilita demonstração local, auditoria e substituição futura de componentes.



Componentes



Processo de Fine-tuning

O fine-tuning usa o dataset PubMedQA como base médica acadêmica. O script de preparação lê `fine-tuning/data/ori_pqa1.json`, transforma cada item em mensagens no formato conversacional e divide os dados em treino e validação com semente fixa.

Preparação dos dados

O arquivo `1_preparar_dataset.py` constrói prompts com contexto, pergunta e resposta esperada, embaralha os exemplos e gera `train.jsonl` e `validation.jsonl`.

Treinamento LoRA

O arquivo `2_treinar_modelo.py` executa `mlx_lm.lora` sobre `mlx-community/Llama-3.2-3B-Instruct-4bit`, salvando o adapter em `fine-tuning/adapters/llama3_2_3b_pubmedqa`.

```
./fine_tuning.sh  
# Etapas principais:  
python fine-tuning/1_preparar_dataset.py  
python fine-tuning/2_treinar_modelo.py
```

O objetivo do adapter é especializar o comportamento do LLM para perguntas biomédicas contextualizadas, mantendo a execução local em Apple Silicon por meio de MLX.

Assistente Médico Criado

O assistente atua como camada intermediária entre linguagem natural e ferramentas estruturadas. Ele decide quando chamar o modelo preditivo, extrai um payload clínico, converte esse payload em 24 features técnicas e interpreta a resposta para o usuário.

Decisão de uso do modelo

O grafo usa termos explícitos, contexto mamário e idade; quando configurado, também chama a LLM para decidir se o Random Forest deve ser consultado.

Extração e transformação

A LLM extrai campos como idade, sexo, raça/cor, mamografia prévia, risco elevado e indicação. O código converte isso para o schema do modelo supervisionado.

Ferramenta preditiva

A tool `prever_risco_cancer_mama` faz chamada HTTP para `ML_API_URL/predict` com as features estruturadas.

Resposta final

O prompt final pede interpretação da classe, da probabilidade da classe positiva e das limitações, sem expor JSON ou features internas ao usuário.

Contrato do endpoint

```
POST http://localhost:8010/chat/messages
```

```
{
```

```
  "message": "Paciente feminina, 54 anos, parda, mamografia  
previa..."
```

```
}
```

Modelo Preditivo Random Forest

A API de ML encapsula o modelo do trabalho anterior, treinado a partir de dados SISCAN/DATASUS. Ela carrega o bundle `ml/model/random_forest.joblib`, preservando nomes das features, classe positiva, coluna alvo e métricas globais.

ACCURACY	RECALL	F1	PRECISION	ROC AUC	BALANCED ACC.
0.760875	0.424779	0.130850	0.077336	0.628885	0.600263

A probabilidade retornada pela API é a estimativa de `predict_proba` para a classe positiva do modelo. Ela não deve ser interpretada como probabilidade clínica diagnóstica real nem como métrica global de acerto.

Hiperparâmetros exportados: `n_estimators=100`, `max_depth=8`, `min_samples_leaf=10` e `class_weight=balanced`.

Segurança, Validação e Explainability

Limites clínicos

O prompt de sistema proíbe diagnóstico definitivo e prescrição. O rodapé obrigatório informa: "Uso acadêmico; não substitui avaliação clínica."

Validação de entrada

O chat usa Pydantic com tamanho máximo de mensagem, papéis permitidos e contratos explícitos para requisição e resposta.

Auditoria

Cada execução registra decisão, payload clínico, features, tool chamada, retorno do modelo, fontes, erros e aplicação do aviso.

Campos registrados no log

- `should_use_predictive_model` e motivo da decisão.
- `clinical_payload` extraído da mensagem do médico.
- `features` enviadas ao Random Forest.
- `tool_result` com predição, probabilidade e métricas.
- `sources` e indicador de aplicação do aviso de segurança.

Interface Web

O frontend foi implementado em Flutter Web como uma aplicação de chat de tela única. Ele consome `POST /chat/messages`, apresenta mensagens do médico e do assistente, permite limpar a conversa e exibe aviso acadêmico fixo na base da tela.

Estrutura

A implementação segue separação por feature, com domínio, repositórios, DTOs, cliente HTTP, presenter e widgets de apresentação.

Distribuição

O build Docker serve o Flutter Web via Nginx e recebe `API_BASE_URL` por argumento de build.

Avaliação e Resultados

A avaliação combina métricas do modelo supervisionado, verificação de integração local e evidências de auditoria. A API de ML expõe `/metadata` com métricas e schema, enquanto o assistant registra os resultados das chamadas em `assistant/logs/audit.jsonl`.

Item avaliado	Resultado observado	Interpretação
Modelo Random Forest	ROC AUC 0.628885 e balanced accuracy 0.600263	Modelo útil como apoio acadêmico exploratório, com limitações importantes devido ao desbalanceamento e baixa precision.
Fluxo automatizado	Casos mamários acionam a tool <code>prever_risco_cancelar_mama</code>	A orquestração conecta texto livre, extração de features, inferência e explicação final.
Explainability	Resposta inclui fonte e significado da probabilidade	O usuário recebe interpretação sem confundir probabilidade do classificador com diagnóstico.
Segurança	Aviso acadêmico obrigatório e logs de auditoria	O sistema registra a trilha de decisão e limita a atuação do assistente.

Execução Local

A entrega inclui scripts para primeira execução, treinamento e subida da aplicação local. O Docker Compose inicia a API de ML e o frontend; o assistant roda localmente para acessar MLX e o adapter.

```
# Primeira execução: fine-tuning + aplicação
cd trabalho3
./primeira_execucao.sh

# Execução da aplicação após adapter treinado
./start_local.sh

# Endpoints
Frontend: http://localhost:8080
Assistant: http://localhost:8010/health
ML API: http://localhost:8000/health
```

Variáveis relevantes

LLM_PROVIDER=mlx

ML_API_URL=http://localhost:8000

EXTRACT_FEATURES_WITH_LLM=true

TRANSLATE_TO_PTBR=true

MLX_ADAPTER_PATH=adapters/llama3_2_3b_pubmedqa

Evidências de Código e Artefatos

- `fine-tuning/data/ori_pqal.json`: dataset médico usado para gerar exemplos de treino e validação.
- `fine-tuning/adapters/llama3_2_3b_pubmedqa/`: adapter LoRA salvo após o treinamento.
- `assistant/app/graph.py`: fluxo LangGraph com decisão, extração, tool call, geração e tradução.
- `assistant/app/tools.py`: conversão de dados clínicos em features e chamada à API preditiva.
- `ml/app/model_service.py`: carregamento do modelo, validação de schema e inferência com `predict_proba`.
- `fe/lib/features/chat/`: implementação da interface de chat em Flutter Web.

Limitações e Próximos Passos

Limitações atuais

Dados e protocolos são acadêmicos; o modelo supervisionado tem baixa precisão; a extração de texto livre pode falhar; a aplicação não integra prontuário real nem autenticação.

Evoluções recomendadas

Adicionar base vetorial de protocolos internos, autenticação, testes end-to-end, avaliação formal da LLM, calibração do modelo e validação clínica supervisionada.

Relatório gerado a partir do enunciado `8IADT - Fase 3 - Tech challenge.pdf` e dos artefatos presentes em `trabalho3/`. Finalidade acadêmica.